



Sistema de Gestión Académica y Planificación Móvil en Tiempo Real

Universidad Autónoma de Chile

Ingeniería Civil Informática

Desarrollo de Aplicaciones Móviles

Integrantes:

Felipe Acevedo

Antonio Arenas

Profesor:

Cristofher Rojas

Fecha:

29 de mayo de 2026

Índice

1. Introducción	3
1.1. Descripción General del Proyecto	3
1.2. Objetivos del Proyecto	3
1.3. Alcance del Proyecto	4
2. Tecnologías Utilizadas	5
2.1. Lenguajes de Programación	5
2.2. Frameworks y Librerías	5
2.3. Entorno de Desarrollo y Control de Versiones	5
3. Modelo de Base de Datos	6
3.1. Diseño de la Base de Datos	6
3.2. Estructura de Colecciones NoSQL	6
3.3. Diagrama del Modelo de Datos	7
4. Descripción de Módulos	8
4.1. Módulo de Usuarios y Autenticación	8
4.2. Módulo Principal (Dashboard Reactivo)	8
4.3. Módulo de Modificación y Control	8
5. Gestión de Roles y Permisos	9
5.1. Tipos de Usuarios (Modelo RBAC)	9
5.2. Permisos y Seguridad en las Reglas de Negocio	9
6. Funcionalidades Avanzadas y Robustez	10
6.1. Validaciones de Datos y Estabilidad del Sistema	10
6.2. Sincronización Asíncrona en Tiempo Real	10
6.3. Implementación chat y notificaciones	10
7. Capturas del Sistema	11
7.1. Interfaz del Módulo de Autenticación	11
7.2. Panel Diario de Tareas y Clases	15
7.3. Anexar tarea o curso para alumno	19
8. Explicación Técnica del Proyecto	23
8.1. Arquitectura de Software Declarativa	23
8.2. Flujo de Operación y Control de Estados	23

9. Mini Manual de Usuario	24
9.1. Primeros Pasos e Inicio de Sesión	24
9.2. Operaciones de Tareas Diarias	24
9.3. Creacion cursos y notas	24
10.Conclusión	25
11.Bibliografía	26

1. Introducción

En el contexto educativo actual, uno de los desafíos más comunes entre los estudiantes es la dificultad para gestionar eficientemente sus compromisos académicos. Pruebas, trabajos, tareas y calificaciones suelen encontrarse dispersos en múltiples plataformas o simplemente se pierden por la ausencia de un sistema de organización centralizado. Esta fragmentación de la información no solo repercute negativamente en el rendimiento académico, sino que también contribuye al aumento del estrés y la sensación de descontrol en el día a día estudiantil.

Frente a esta realidad, surge la necesidad de contar con una herramienta digital diseñada específicamente para las necesidades del estudiante moderno. Por ello, se propone el desarrollo de una aplicación móvil orientada a la organización y gestión académica personal. A través de una interfaz accesible e intuitiva, la aplicación integrará funcionalidades clave como la visualización de calificaciones, un calendario académico con recordatorios de evaluaciones y fechas de entrega, y un sistema de registro de tareas y actividades pendientes. El propósito central de esta iniciativa es brindar a los estudiantes una plataforma unificada que simplifique su planificación diaria, fomente hábitos de organización y les permita asumir un control más consciente y efectivo de sus responsabilidades académicas, contribuyendo así a una mejor experiencia de aprendizaje.

1.1. Descripción General del Proyecto

El proyecto consiste en una plataforma móvil multiplataforma nativa que centraliza la administración del tiempo y las responsabilidades curriculares de los estudiantes universitarios. Utilizando una arquitectura reactiva, la aplicación coordina dos flujos de información críticos: agendas diarias y control de asignaturas. Los usuarios pueden interactuar interactivamente mediante vistas de página segmentadas por días, ofreciendo un desglose limpio de los quehaceres inmediatos para evitar la sobrecarga cognitiva común en la educación superior.

1.2. Objetivos del Proyecto

Objetivo General:

Desarrollar una aplicación móvil multiplataforma que permita a los estudiantes gestionar de manera organizada sus notas, tareas, ramos y evaluaciones académicas mediante una arquitectura de software reactiva y persistencia de datos sincronizada en tiempo real.

Objetivos Específicos:

- Diseñar una interfaz intuitiva aplicando patrones de diseño limpio y una paleta de colores armónica para disminuir la fatiga visual.
- Implementar un motor de sincronización de datos en la nube mediante un backend no relacional para asegurar la disponibilidad de la información.
- Desarrollar una lógica de validación estricta para la captura de datos temporales (horarios), evitando estados inconsistentes y caídas en la ejecución (*crashes*).
- Construir un módulo modularizado de segmentación por control de accesos basado en roles (*Role-Based Access Control*).
- Proporcionar mecanismos de persistencia desacoplados para agilizar futuras ampliaciones de módulos funcionales.

1.3. Alcance del Proyecto

El desarrollo del software abarca el despliegue del frontend móvil completo enfocado en dispositivos Android mediante el uso de recursos nativos del SDK. Incluye los módulos de autenticación segura por correo electrónico y OAuth 2.0 con Google, gestión CRUD (Crear, Leer, Actualizar, Borrar) de la colección de tareas personales ordenadas cronológicamente, y un lector reactivo para los horarios asignados de los ramos del período en curso. El sistema opera de manera conectada, requiriendo conexión activa a la red para la sincronización con el clúster de Firebase.

2. Tecnologías Utilizadas

El desarrollo técnico del sistema se sustenta en herramientas modernas de desarrollo de software que optimizan los tiempos de compilación y aseguran una ejecución fluida.

2.1. Lenguajes de Programación

- **Dart:** Lenguaje optimizado para la construcción de interfaces de usuario rápidas en múltiples plataformas. Su compilación AOT (*Ahead-of-Time*) permite un rendimiento de código de máquina nativo, eliminando cuellos de botella en la renderización gráfica.

2.2. Frameworks y Librerías

- **Flutter:** Framework de UI declarativo que permite construir aplicaciones compiladas nativamente a partir de un solo código base.
- **material.dart:** Estándar de diseño y componentes visuales para garantizar una experiencia táctil intuitiva.
- **firebase_core y cloud_firestore:** APIs esenciales encargadas de la comunicación con los servicios del backend no relacional en la nube.
- **firebase_auth:** Gestor de sesiones seguro encargado de encriptar e intermediar los tokens de acceso de los usuarios.

2.3. Entorno de Desarrollo y Control de Versiones

- **Android Studio / VS Code:** IDEs principales configurados con herramientas de depuración (*Flutter DevTools*) para supervisar el árbol de widgets y el consumo de memoria en emuladores.
- **Git y GitHub:** Control de versiones distribuido utilizado para asegurar la trazabilidad del código fuente mediante commits estructurados, protegiendo las ramas estables del software.

3. Modelo de Base de Datos

3.1. Diseño de la Base de Datos

La persistencia de datos del sistema se estructuró sobre Firestore, una base de datos NoSQL que a diferencia de un modelo relacional tradicional basado en tablas fijas, el esquema arquitectónico se organiza mediante colecciones que agrupan documentos flexibles compuestos por pares de clave-valor.

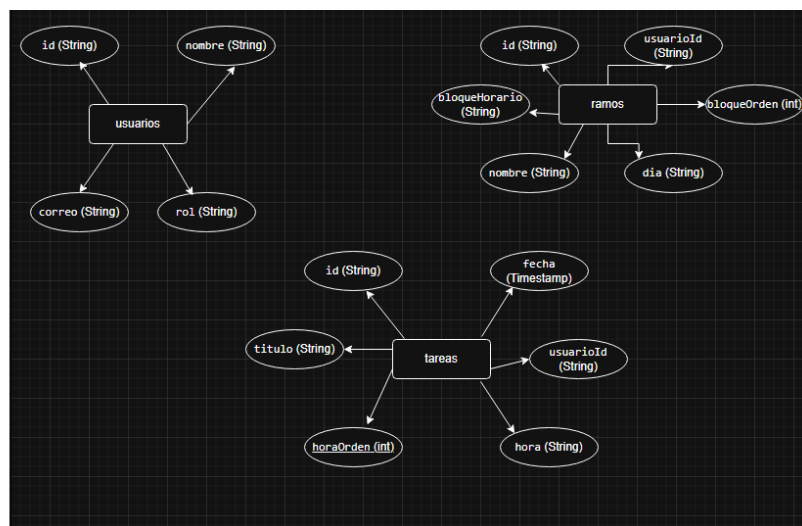
3.2. Estructura de Colecciones NoSQL

A continuación, se detalla la estructura lógica de los documentos almacenados en la base de datos para garantizar búsquedas eficientes e indexadas:

Listing 1: Esquema lógico de Colecciones en Firestore

```
{
  "usuarios": {
    "nombre": "String",
    "correo": "String",
    "rol": "String"
  },
  "tareas": {
    "usuarioId": "String",
    "titulo": "String",
    "hora": "String",
    "horaOrden": "Integer",
    "fecha": "Timestamp"
  },
  "ramos": {
    "usuarioId": "String",
    "nombre": "String",
    "dia": "String",
    "bloqueHorario": "String",
    "bloqueOrden": "Integer"
  }
}
```

3.3. Diagrama del Modelo de Datos



4. Descripción de Módulos

4.1. Módulo de Usuarios y Autenticación

Este módulo se encarga de resguardar el acceso restringido a la aplicación. Interpola el estado de autenticación global provisto por Firebase. Si el usuario no cuenta con una sesión activa, el sistema conmuta la vista automáticamente hacia las pantallas de registro o recuperación de contraseña, impidiendo el renderizado de la información del negocio hasta validar los tokens de seguridad.

4.2. Módulo Principal (Dashboard Reactivo)

Es el núcleo funcional para el estudiante. Integra un componente de control de páginas (‘PageView’) vinculado a selectores táctiles diarios. Este módulo lee continuamente los cambios realizados en las tareas o ramos del día específico seleccionado, actualizando la pantalla del dispositivo móvil al instante con asíncronas de datos.

4.3. Módulo de Modificación y Control

Administra las operaciones transaccionales complejas de la aplicación, como los diálogos emergentes personalizados (‘AlertDialog’) utilizados para agregar, editar o eliminar registros de tareas de la nube de forma segura mediante métodos ID del documento de Firebase.

5. Gestión de Roles y Permisos

5.1. Tipos de Usuarios (Modelo RBAC)

El sistema implementa una lógica de control de accesos basada en roles segmentados:

- **Rol Alumno:** Perfil consumidor primario. Tiene acceso completo a la visualización de su horario de clases personalizado y control total de sus tareas individuales mediante operaciones CRUD.
- **Rol Profesor:** Perfil administrador de contenido académico. Posee facultades extendidas orientadas al despliegue de planificaciones y gestión centralizada de ramos asociados.
- **Rol Admin:** Perfil administrador de contenido relacionada a la app, controla los usuarios y los datos de cada persona junto a gráficos de estadísticas y acceso a un chat de soporte.

5.2. Permisos y Seguridad en las Reglas de Negocio

Los permisos están validados en dos capas, la primera es la interfaz de usuario con las verificaciones correspondientes a la sección necesaria, ocultando los componentes de administración (como botones de adición globales) según el rol leído durante el inicio de sesión. La segunda capa es en el servidor de Firebase mediante reglas .

6. Funcionalidades Avanzadas y Robustez

6.1. Validaciones de Datos y Estabilidad del Sistema

Para evitar caídas de ejecución críticas debido al procesamiento de datos corruptos, la aplicación implementa motores de validación basados en Expresiones Regulares estrictas antes de comprometer los cambios con la base de datos:

$$ExpresinRegulardeTiempo = ^{([0-1]?[0-9]|2[0-3]):[0-5][0-9]}$ \quad (1)$$

Esta expresión puede ser cambiada a un anexo de fecha fijo, el cual se vera segun el avance del proyecto su implementacion.

6.2. Sincronización Asíncrona en Tiempo Real

La aplicación utiliza flujos continuos de Streams abiertos a firestore que hace que todo sea reactivo y casi instantaneo.

6.3. Implementacion chat y notificaciones

La aplicación utiliza un chat cerrado entre alumno y profesor, junto a notificaciones cuando el profesor anexa algo a sus alumnos, la idea es agrandar el chat para que se pueda enviar archivos y asignar mejor las tareas.

7. Capturas del Sistema

7.1. Interfaz del Módulo de Autenticación

Vistas de acceso del sistema que incorporan la recuperación y blanqueo de credenciales mediante el envío automatizado de correos electrónicos transaccionales securizados.

12:37

<



PROFESOR

Nombre Completo

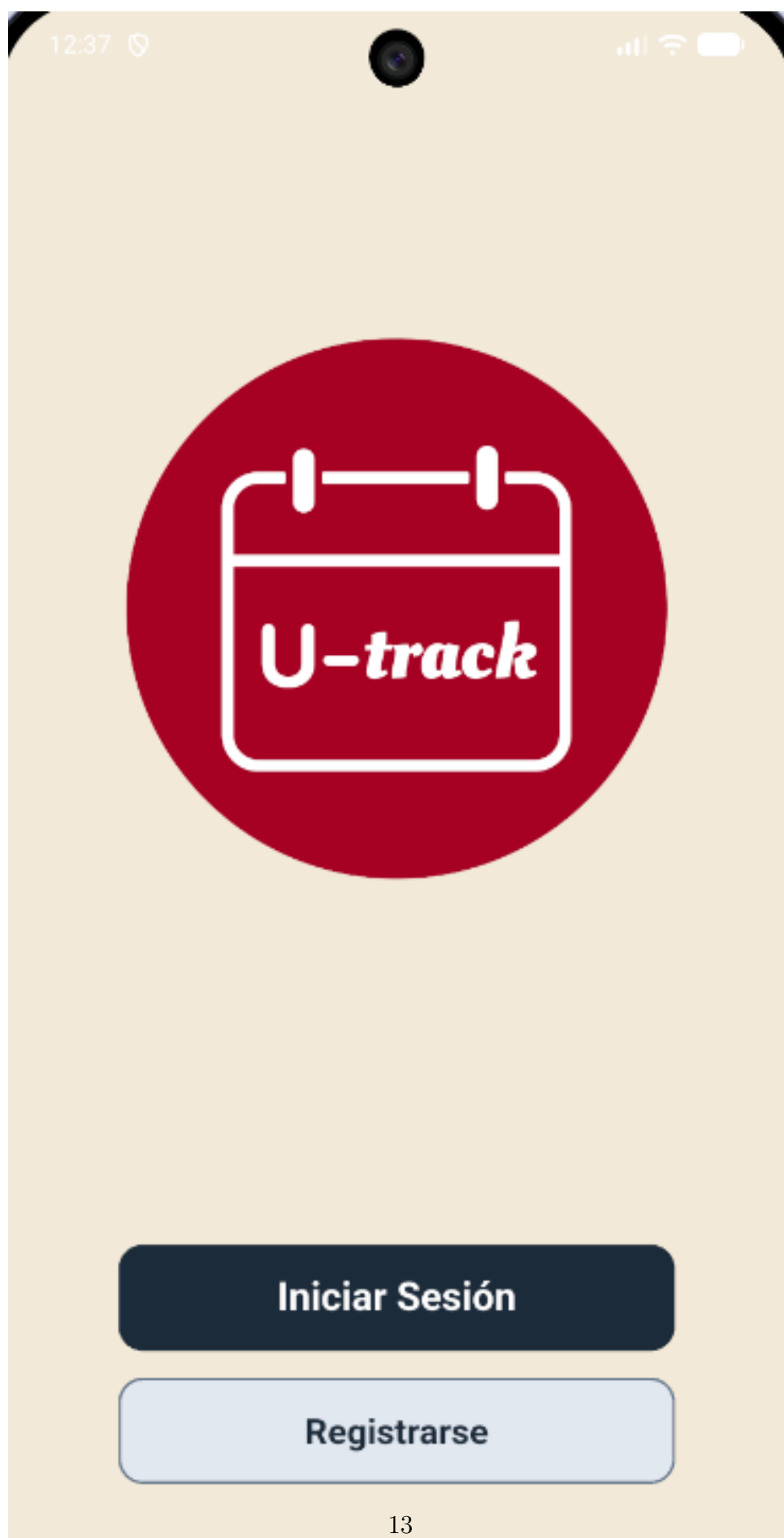
 **Felipe acevedo**

 Correo Electrónico
felipeacevedocarrasco@gmail.com

 Miembro Desde
7/5/2026

Guardar Cambios

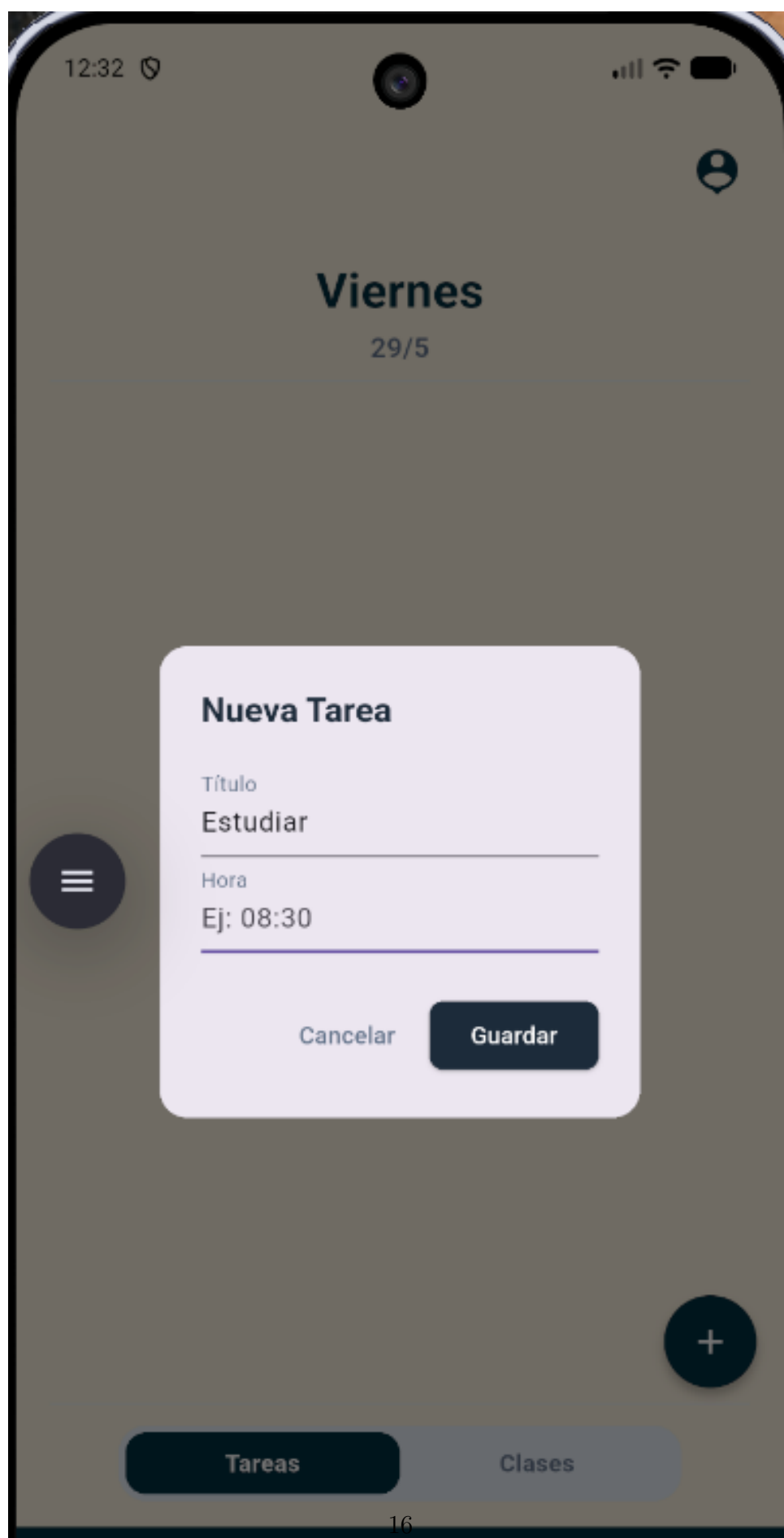
Cerrar Sesión

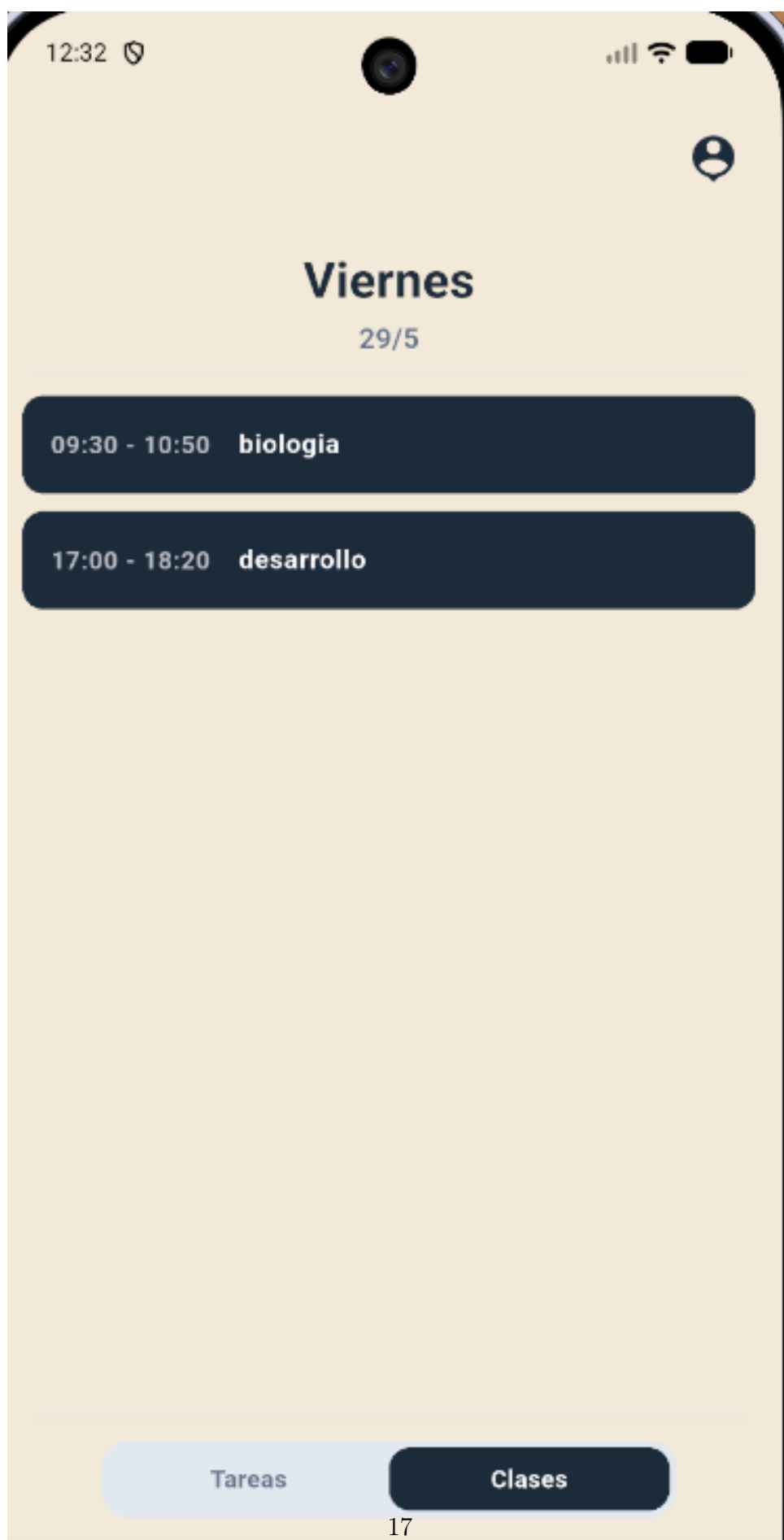


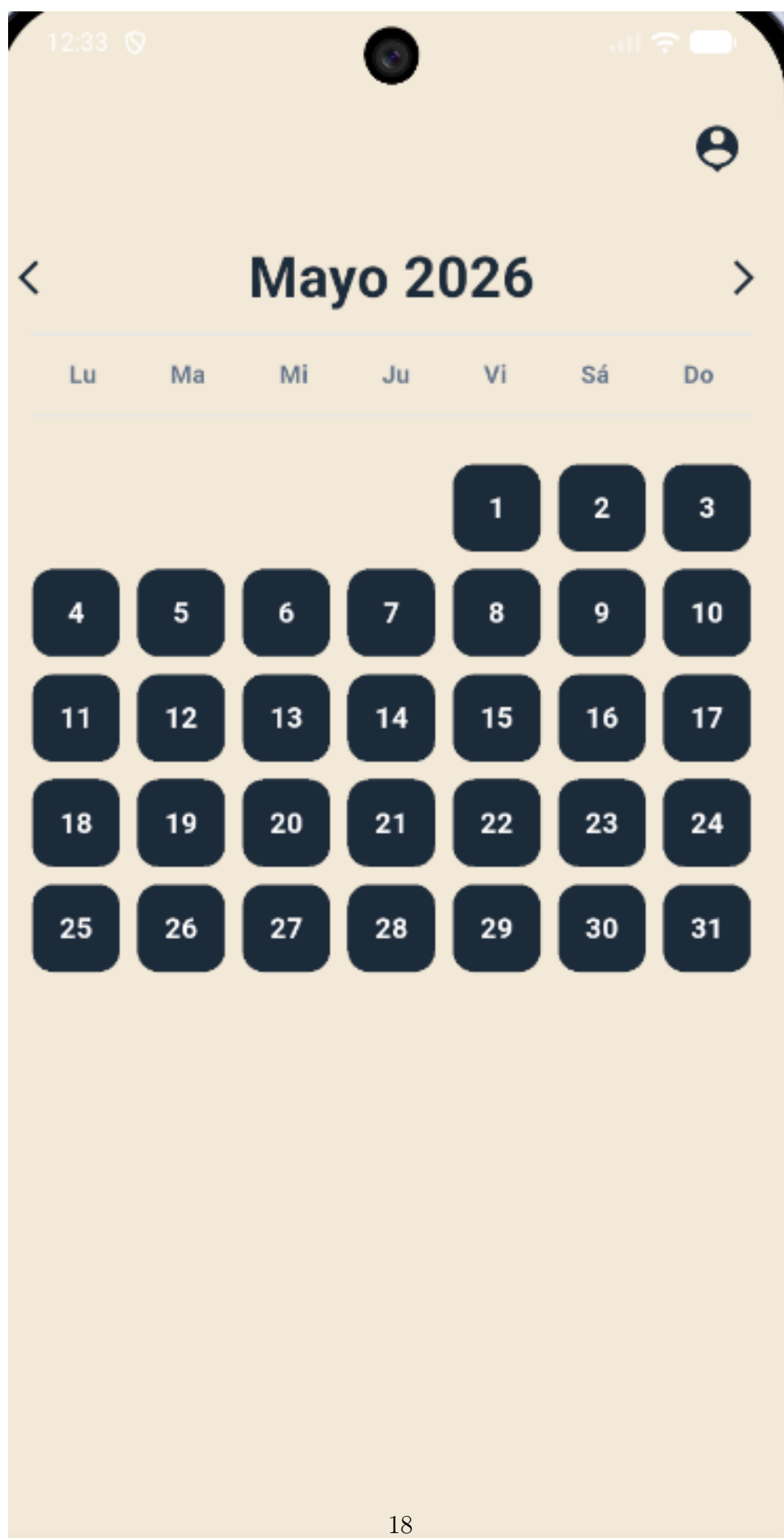


7.2. Panel Diario de Tareas y Clases

Pantallas donde se exponen las tarjetas cronológicas diseñadas con bordes curvos y tipografías legibles bajo el esquema de color corporativo de la solución.

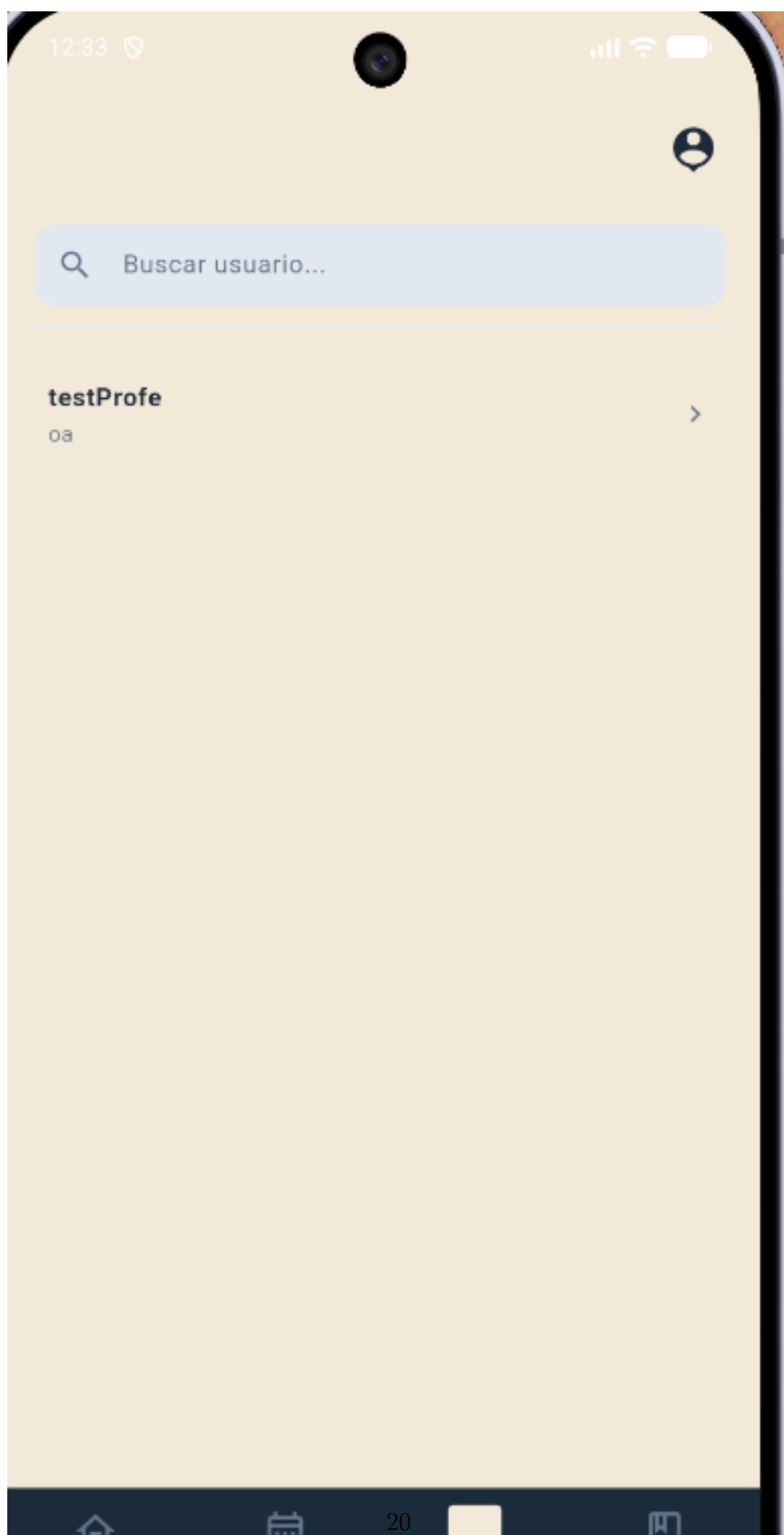


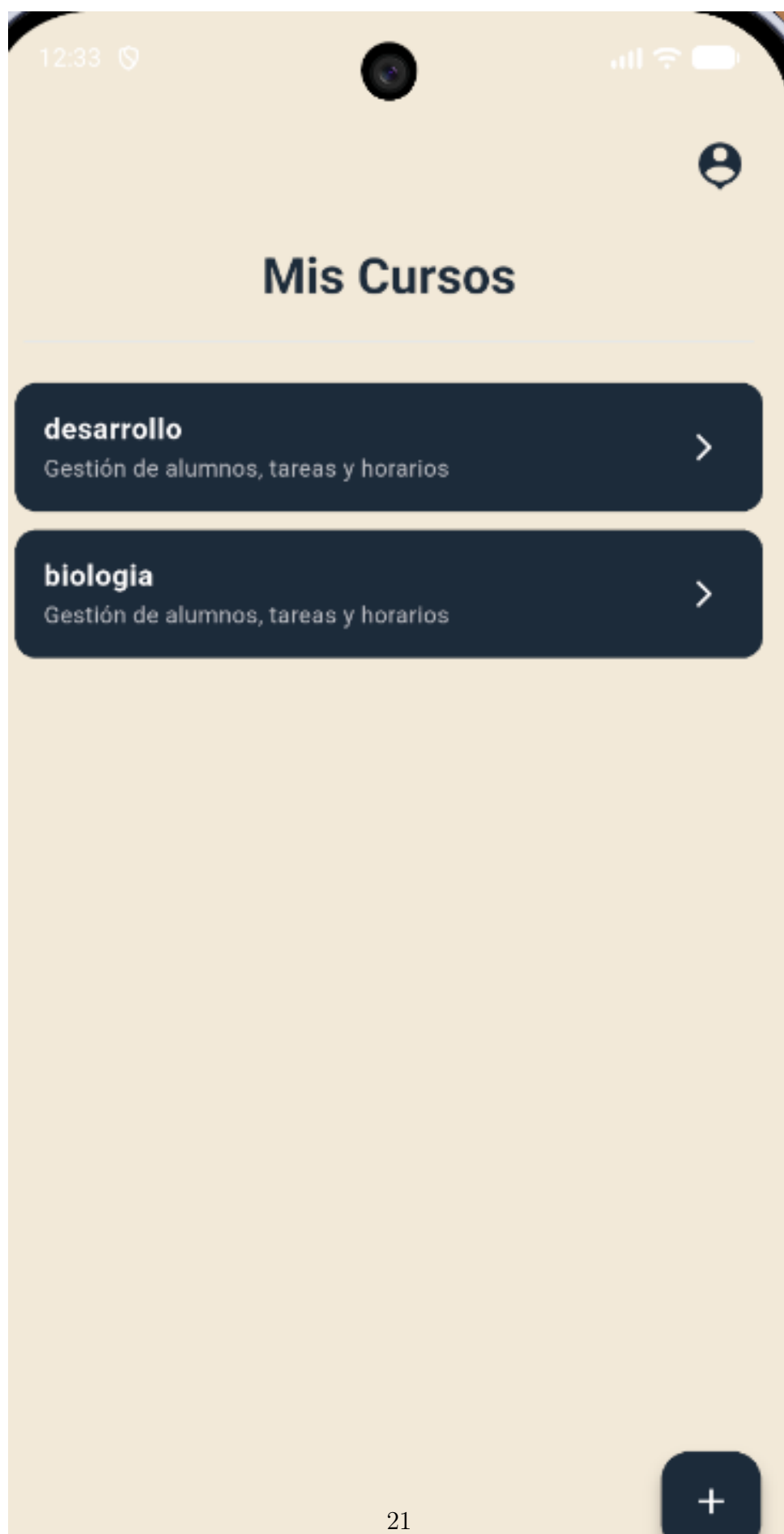


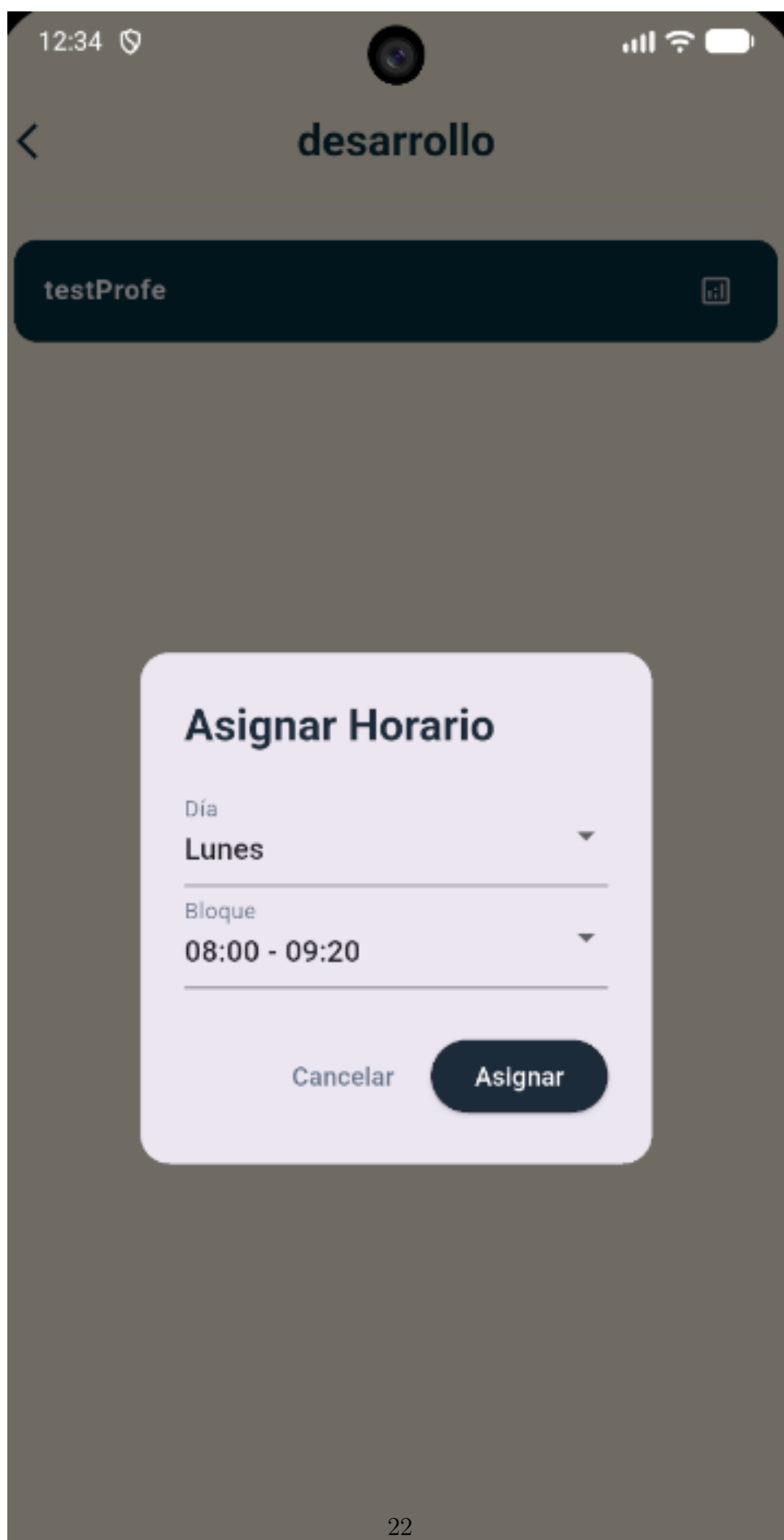


7.3. Anexar tarea o curso para alumno

Pantallas donde se exponen las tarjetas cronológicas diseñadas con bordes curvos y tipografías legibles bajo el esquema de color corporativo de la solución.







8. Explicación Técnica del Proyecto

8.1. Arquitectura de Software Declarativa

La aplicación es reactiva de Flutter, donde la interfaz gráfica se actualiza constantemente en la app. La pantalla responde dinámicamente a los cambios detectados en las consultas asíncronas remotas.

8.2. Flujo de Operación y Control de Estados

El siguiente bloque esquematiza el método transaccional utilizado para resguardar la consistencia de datos durante los procesos de actualización de tareas existentes dentro del ciclo de vida del Widget:

```
onPressed: () async {  
  if (tareaCtrl.text.isNotEmpty && regex.hasMatch(tareaCtrl.  
    text.trim())) {  
    List<String> partes = tareaCtrl.text.split(":");  
    int o = (int.parse(partes[0]) * 100) + int.parse(partes  
      [1]);  
  
    await FirebaseFirestore.instance.collection("tareas").doc(  
      doc.id).update({  
        "titulo": tareaCtrl.text.trim(),  
        "hora": tareaCtrl.text.trim(),  
        "horaOrden": o,  
      });  
    if (context.mounted) Navigator.pop(context);  
  }  
}
```


9. Mini Manual de Usuario

9.1. Primeros Pasos e Inicio de Sesión

1. Al abrir la aplicación, el usuario debe introducir sus credenciales validadas o bien presionar el ícono de autenticación unificada con Google.
2. En caso de olvido de la clave de acceso, debe seleccionar el enlace "¿Olvidaste contraseña?", escribir su correo registrado y seguir las instrucciones contenidas en el mensaje de recuperación enviado automáticamente a su casilla.

9.2. Operaciones de Tareas Diarias

Para Crear hay desplazamiento con gestos laterales al día deseado en la pestaña "Tareasz presione el botón flotante para agregar Para Editar o Eliminar directamente sobre la tarjeta de la tarea que desea gestionar; emergerá un panel flotante de decisión para modificar los valores o eliminarlos.

9.3. Creacion cursos y notas

Se puede crear una sala y agregar alumnos si el rol es "profesor" de manera que a todos los alumnos asociados al crearles una tarea o horario curso se le actualizara al alumno y notificara.

10. Conclusión

El desarrollo de esta plataforma demuestra la viabilidad y eficiencia de combinar frameworks declarativos de alto nivel como Flutter con infraestructuras de bases de datos no relacionales en tiempo real como Firebase para mitigar problemáticas organizacionales reales del estudiantado.

La implementación exitosa de validaciones complejas mediante expresiones regulares, la estructuración de colecciones NoSQL legibles y escalables, y el desacoplamiento efectivo de los permisos basados en roles de usuario sientan un precedente informático robusto. El software final no solo actúa como un asistente organizativo directo para mitigar la deserción y la sobrecarga académica, sino que cumple cabalmente con las normativas y metodologías de ingeniería de software necesarias para ser considerado un producto de alta disponibilidad, escalable y seguro.

11. Bibliografía

- **Flutter Documentation:** *Build desktop, mobile, and web apps from a single codebase.* URL: <https://docs.flutter.dev/>
- **Firebase Firestore Documentation:** *Cloud Firestore - Store and sync data with a NoSQL cloud database.* URL: <https://firebase.google.com/docs/firestore>
- **Google Sign-In for Flutter:** *Authentication services for mobile platforms.* URL: https://pub.dev/packages/google_sign_in